SIH1454 · NTRO · BLOCKCHAIN & CYBERSECURITY · SOFTWARE

PHISHEYE

End-to-End URL Scanning Engine & Threat Detection Architecture

A publication-grade reference for the FastAPI scanning core: ingestion, SSRF-safe expansion, public threat feeds, lookalike heuristics, psychological language analysis, OpenCV QR handoff, weighted fusion, and persistence.

PRODUCT PHISHEYE API v2.0.0	**RUNTIME** FastAPI · Uvicorn · Python 3
CLIENTS React 19 / Vite · Chrome MV3	**STORE** SQLite *threats.db*

Shipped behaviour is labelled **Live**. Production extensions that are designed but not running in this repository are labelled **Roadmap**. This document does not invent a trained CNN, Redis cluster, or live PostgreSQL.

Contents

1. Executive Architecture Overview & Request Lifecycle

2. Ingestion, Normalization & Security Safeguards

3. Domain & Infrastructure Footprint Analysis

4. Public Threat Intelligence Integrations

5. Heuristic & Behavioral Intelligence Engine

6. Weighted Fusion Engine & Decision Scoring

7. Data Persistence & Production Readiness

SECTION 01

Executive Architecture Overview & Request Lifecycle

PHISHEYE is a **modular monolith**. One FastAPI process on port `8000` is the threat engine. The React dashboard (`127.0.0.1:5173`) and the Manifest V3 extension are presentation clients. They share `POST /api/v1/threat/check-url` and the multi-channel `POST /api/v1/analyze/*` family. There is no separate API gateway microservice in the current tree.

Live control plane

- **ASGI:** Uvicorn serving `app.main:app` .
- **Validation:** Pydantic v2 request bodies.
- **Auth:** passlib bcrypt + python-jose JWT `HS256` . Claim `sub` = username. Signup/login issue a 7-day bearer token. URL checks may be anonymous.
- **CORS:** `allow_origins=["*"]` — prototype, not production hardening.
- **Startup:** `init_db()` then a daemon thread warms OpenPhish / URLhaus / PhishTank / Phishing Army unless `PHISHEYE_SKIP_FEED_WARMUP=1` .

Pipeline slogan

INPUT → NORMALIZE → ANALYZE → FUSE → EXPLAIN → ACT

Simple View (plain English) and Technical View (evidence, providers, tags) are built in `warnings.py` . The extension *gates* navigation so a destination is scored before it is used as the primary surface.

Authentication (JWT HS256)

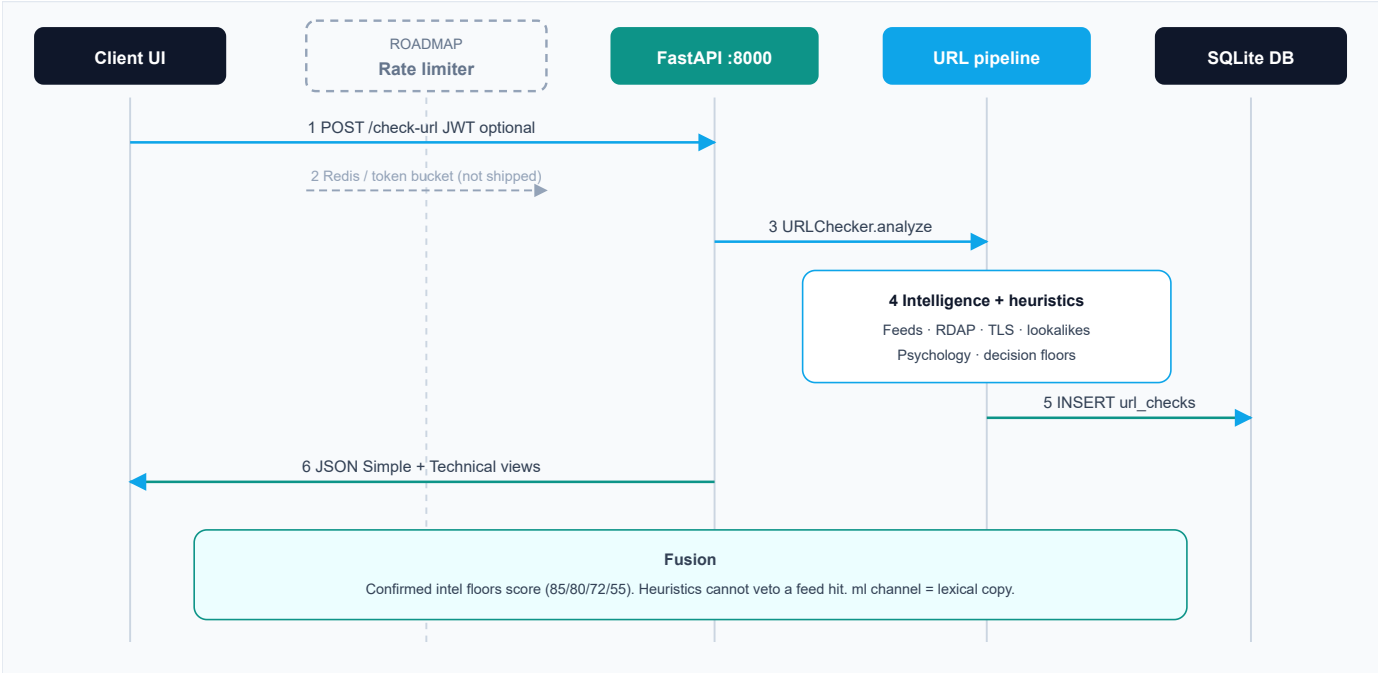
Routes under `/api/v1/user` hash passwords with bcrypt. Protected actions (community report, scan delete, profile) require `Authorization: Bearer <jwt>` . Analyze routes accept an *optional* bearer so a logged-in user is attached to `content_scans.user_id` . Default `SECRET_KEY` is a development fallback and must be rotated for any shared deployment.

Rate limiting — live vs roadmap

LIVE Community reports: one URL per user per 24 hours, 20 reports per day (`reputation.add_report`). Upload bodies capped at 5 MB. Intel HTTP calls use short timeouts (3–8s).

ROADMAP A dedicated gateway with Redis (or in-process token bucket) in front of `/check-url` and `/analyze/*` is **not implemented**. There is no global FastAPI rate-limit middleware and no Redis dependency in `requirements.txt` . The sequence diagram below shows the live path in solid ink and the limiter as a dashed production insert.

FIGURE 1 — REQUEST LIFECYCLE (LIVE PATH SOLID; RATE LIMITER DASHED / ROADMAP)



Primary JSON contract

```
POST /api/v1/threat/check-url
{
  "url": "https://sbi-login.xyz/verify",
  "view": "both",
  "page_signals": { "buttons": 0, "iframes": 0, "popups": 0, "overlays": 0, "links": 0 }
}

→ risk_score, risk_level, classification, safe,
  simple_view, technical_view, evidence, debug, threat_tags
```

SECTION 02

Ingestion, Normalization & Security Safeguards

Scan logic gateway

Ingestion is not a separate process. FastAPI routers are the gateway:

Input	Route	Normalizer
Plain URL	`POST /api/v1/threat/check-url`	Scheme prepend `https://` if missing → `URLChecker.analyze`
Pasted text / SMS / WhatsApp / email	`POST /api/v1/analyze/content`	`content_normalizer` + `url_extractor` (multi-link batch from one body)
QR image	`POST /api/v1/analyze/qr`	OpenCV decode → URL strings → same content pipeline
Screenshot	`POST /api/v1/analyze/screenshot`	QR always; OCR only if Tesseract is installed; optional hint URL
Browser tab / gated navigation	Extension → same `check-url`	Identical engine, different client

A “batch” in the live product is **N URLs extracted from one message**, each passed through `URLChecker`, not a bulk CSV job API.

URL normalization & expansion

`url_normalize.normalize_url` never fetches. It:

- Accepts bare hosts and forces `http` / `https` only.
- Strips userinfo (`user:pass@host`), default ports 80/443, and `#fragments` .
- Keeps path (so `/login` ≠ `/`) and stores query separately (so `google.com/?q=phish` is not the homepage).
- Computes hostname and registrable domain, including multi-part TLDs such as `sbi.co.in` .
- IDNA-encodes the host (`host.encode("idna")`) so punycode/Unicode labels become a stable ASCII key.

`redirects.expand_url` unwraps known shorteners (`bit.ly` , `t.co` , `tinyurl.com` , `cutt.ly` , …) hop-by-hop (max 8), HEAD then GET-on-405, after SSRF checks. Scoring: +12 if expanded, +20 if the destination stays hidden, +10 if hops > 2. The *final* URL is what intel and heuristics see.

SSRF mitigation guards

`ssrf.assert_public_http_url` runs before any outbound fetch of an attacker-controlled URL (shortener unwrap, optional ad/clutter fetch). Threat-intel matching **does not fetch the submitted phishing page**.

Blocked hosts

- `localhost` , `*.localhost` , `*.local`
- `metadata.google.internal` , `metadata` , `instance-data`
- `0.0.0.0`

Blocked addresses

- Loopback `127.0.0.0/8`
- RFC1918 `10.0.0.0/8` , `172.16.0.0/12` , `192.168.0.0/16`
- Link-local, multicast, reserved, unspecified
- Cloud metadata `169.254.169.254` , `fd00:ec2::254`

Implementation uses `ipaddress` predicates plus `socket.getaddrinfo` so a public-looking hostname that resolves to a private IP is also rejected (`UnsafeURLError`).

SECTION 03

Domain & Infrastructure Footprint Analysis

DNS resolution & record validation

LIVE `external_intel.check_domain_exists` performs `socket.getaddrinfo(host)` (3s timeout). That covers A/AAAA reachability as “does this name resolve?” NXDOMAIN → tag `domain_not_found`, heuristic +35, and `safe` cannot be true.

ROADMAP Explicit MX / TXT collection for SPF, DKIM, and DMARC on the *sending* domain is not a current URLChecker stage. Those records matter for email authentication; the live email path only scores caller-supplied From / Reply-To against brand domain lists in `sender_analyzer.py`.

Domain age & registrar verification

LIVE Age comes from public **RDAP** (`rdap.org/domain/{name}`, plus Verisign RDAP for `.com` / `.net`), not a paid WHOIS parser. Registration event dates yield `domain_age_days`. Scoring: < 30 days +20 (NRD), < 90 days +10. Cache is in-process.

ROADMAP Full registrar, nameserver, and privacy-proxy WHOIS fields; automated takedown ticketing.

TLS / SSL inspection

`ssl_certs.inspect_tls` is SSRF-gated, 5s timeout. A padlock is recorded, never treated as “this is SBI.”

Check	Live behaviour	Risk points (cap 40)
HTTP URL	Status `http`, tag `no_https`; URLChecker also +5 for missing https	scheme extra in checker
Trusted chain + hostname	Default SSL context + SNI `server_hostname=host`	0
Hostname / SNI mismatch	Status `hostname_mismatch`	+30
Self-signed / untrusted issuer	Issuer vs subject heuristics + verify error	+20 / +18
Expired cert	Parsed `notAfter`	+25
Expires ≤ 7 days	Warning finding	+8

ROADMAP Dedicated wildcard-abuse policy (e.g. `*.tk` issued to a lookalike) as its own detector. Today wildcard names appear in SAN lists but are not a separate scoring rule.

SECTION 04

Public Threat Intelligence Integrations

Module `threat_intel.py` + cache `feed_cache.py`. Feeds persist under `backend/data/feeds/` (TTL 30 minutes, academic User-Agent). Providers are **independent**: a miss is `no_match`, a timeout is `unavailable`. Neither is rewritten to SAFE. Confirmed hits cannot be averaged away by heuristics (`decision_engine.py`).

Feed	How it is queried	What a hit means	Weighting in the live engine
Google Safe Browsing v4	Optional `GOOGLE_SAFE_BROWSING_API_KEY`. POST `safebrowsing.googleapis.com/v4/threatMatches:find` with types SOCIAL_ENGINEERING, MALWARE, UNWANTED_SOFTWARE, POTENTIALLY_HARMFUL_APPLICATION. v5 is not wired.	API match → `confirmed_malicious`. Missing key → provider skipped, not a fake clean.	Feeds into intel overall status. Decision floors 55–85 by match type (see §6). Fusion channel `threat_intel` weight **0.10** on the content path; URL path uses floors, not that weight alone.
PhishTank	Public CSV dump `online-valid.csv`. Without `PHISHTANK_API_KEY` only **verified** rows are used. Key enables live lookup of unverified reports.	Verified listing → confirmed. Unverified → `reported_malicious` (floor 58), not a court proof.	Same intel contract. Path/query-aware matching so one dump row cannot burn all of YouTube or Google.
URLhaus (abuse.ch)	Public online text feed + optional `URLHAUS_AUTH_KEY` for API. Cached locally.	URL listed as malware/botnet payload host → confirmed.	Independent provider row in `evidence.threat_intelligence.providers`.
OpenPhish	`https://openphish.com/feed.txt` pulled on warmup / TTL. Indexed by raw, normalized URL, and host.	Exact / normalized / (restricted) path match → confirmed phishing telemetry.	Trusted-domain generic paths (`/watch` , `/search` , `/`) require query overlap.
Phishing Army	Domain blocklist `phishing.army/download/phishing_army_blocklist.txt`.	Hostname / registrable-domain listing.	**Skipped on official trusted destinations** (google.com, github.com, Indian banks, …) so a noisy domain list cannot mark the real brand. User-content hosts (`sites.google.com` , `github.io`) stay fully scanned.

Match types, strictest first: exact raw URL → normalized URL → path-related → hostname → registered_domain. Intel floor for exact_url is 85; registered_domain is 55.

SECTION 05

Heuristic & Behavioral Intelligence Engine

Typosquatting & homoglyph engine

LIVE `URLChecker._detect_brand` compares the registrable *label* to a **curated brand map** (PayPal, Amazon, Google, YouTube, GitHub, Microsoft, Apple, Meta brands, Netflix, Flipkart, Paytm, PhonePe, SBI, HDFC, ICICI, Axis, IRCTC — not the Tranco top 10k). Pipeline:

1. Skip if the domain already *is* the official brand host.
2. Normalize leetspeak (`1→l` , `0→o` , `@→a` , …) and a limited Cyrillic→Latin homoglyph table.
3. Substring brand hit *or* Levenshtein distance 0–2 with similar length → tag `brand_impersonation` , typically +40.

Known fragments (`paypa1` , `amaz0n` , `go0gle` , `bank-login`) add +40 independently. Free-app hosts (`vercel.app` , `lovable.app` , `pages.dev` , `github.io` , …) plus login/otp/bank tokens add +22 to +34.

ROADMAP Levenshtein / homograph screening against a top-10k domain list and a fuller IDN confusable table (Unicode TR39).

NLP & psychological triggers

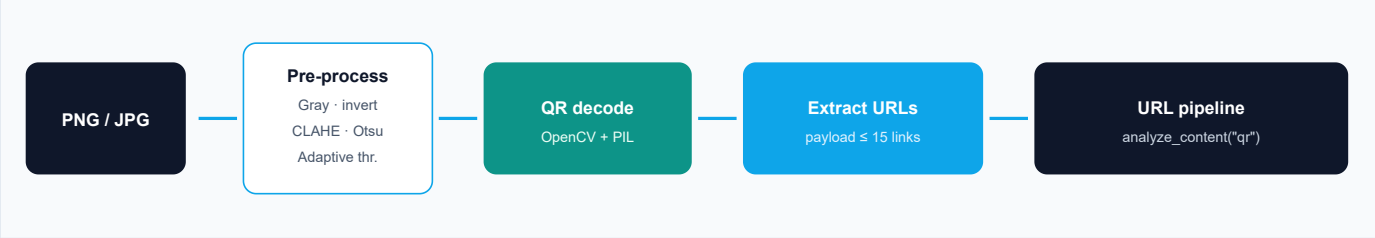
This is **not** a neural NLP model. `psychology.py` is deterministic phrase matching (English + Hinglish): urgency, fear, authority, coercion, financial pressure, OTP/credential harvest, countdown, greed/lottery, secrecy. Examples: *enter otp*, *account blocked*, *abhi click*, *jaldi karo*, *aadhaar otp*.

`brand_detector.detect_impersonation` requires brand mention *and* unofficial domain *and* an extra scam signal — a news sentence that says “SBI” is not enough.

False-positive guard: if psychology is the only strong channel, fused score is capped at **35** (stays LOW/MEDIUM). Marketing “act now” cannot solo-CRITICAL.

QR & media extraction (OpenCV)

FIGURE 2 — QR DECODE VARIANTS BEFORE URL HANDOFF (DESTINATION IS NEVER FETCHED)



`qr_analyzer.extract_qr_urls` runs OpenCV `QRCodeDetector` on colour, grayscale, invert, Gaussian blur, CLAHE, Otsu, morphological close, and adaptive Gaussian threshold variants, with optional upscale and a Pillow fallback. Payloads are parsed for `http(s)` or bare hosts (capped at 15 URLs) and handed to `analyze_content(source_type="qr")` . Images stay in memory (max 5 MB).

SECTION 06

Weighted Fusion Engine & Decision Scoring

Two cooperating layers

URL path — decision engine

Single-URL scans finalize in `calculate_final_risk`. Confirmed intel *floors* the score. Heuristics and the lexical "ml" copy cannot override a feed hit.

Message path — weighted fuse

`risk_engine.fuse` combines ten channels. If any child URL was confirmed malicious, the fused score is raised to at least that URL's decision score.

Conceptual aggregation

Architecturally, independent evidence collapses to three families — intel, heuristics/behaviour, network/domain metadata:

$$\text{RiskScore} = w_{intel} \cdot S_{intel} + w_{heur} \cdot S_{heur} + w_{net} \cdot S_{net}$$

The **shipped** content fusion is the ten-term expansion (weights sum to 1.0; starting defaults, not proven-optimal):

```
0.22 url + 0.16 domain + 0.10 ml + 0.10 nlp + 0.10 psychological + 0.10 threat_intel + 0.08 brand + 0.05 sender + 0.05 community + 0.04 redirect
```

The channel named `ml` is a **copy of the lexical/heuristic URL score**. `backend/app/ml/` contains no pickle, no TensorFlow graph, and no sklearn classifier. It exists so a low "ML" number cannot be cited as proof of safety when intel has already confirmed malice.

`model_confidence = min(95, 35 + 10 × count(signals ≥ 20))` — agreement, not detection accuracy. `scam_risk` is the same 0–100 integer, not a calibrated fraud probability.

Intel floors (URL decision engine)

Situation	Floor / rule	Classification
Confirmed · exact URL	max(heuristic, 85)	CONFIRMED_MALICIOUS
Confirmed · normalized URL	max(heuristic, 80)	CONFIRMED_MALICIOUS
Confirmed · hostname	max(heuristic, 72)	CONFIRMED_MALICIOUS
Confirmed · registered domain	max(heuristic, 55)	CONFIRMED_MALICIOUS
Unverified report	max(heuristic, 58)	SUSPICIOUS
Heuristic ≥ 50, no confirm	heuristic	SUSPICIOUS
Unavailable or no_match + weak heuristics	heuristic	NO_MALICIOUS_EVIDENCE — text states this is not a clean bill of health

Verdict categories — live engine

These cut-points are implemented in `decision_engine._level` and `risk_engine._level`. They are the numbers a jury will see on screen.

LOW

0 – 29

No strong malicious indicators. Trusted destinations often land here. Still not a cryptographic identity proof.

MEDIUM

30 – 49

Fishy heuristics, young domain, unverified report, or mixed signals. Extension asks before opening.

HIGH

50 – 69

Strong lookalike / free-host / heuristic cluster or reported intel. Do not enter OTP.

CRITICAL

70 – 100

Confirmed feed hit (floored) or stacked impersonation. Extension keeps the tab closed.

ROADMAP Alternate bandings (for example 0–19 / 20–49 / 50–79 / 80–100) are a product-calibration choice. Shipping two scales at once would contradict the UI. Recalibrate only with labelled evals, then change both Python helpers and this document together.

SECTION 07

Data Persistence & Production Readiness

Current state

Live store is a single SQLite file `backend/threats.db` opened by `sqlite3.connect` with `PRAGMA foreign_keys=ON` and `Row` dict access. Path is anchored to the backend directory regardless of process cwd. All writes use parameterized `?` placeholders.

`sqlalchemy` is listed in `requirements.txt` and declarative models exist under `backend/app/models/` (`User`, etc.). **Runtime routes do not use the ORM.** Saying “we query through SQLAlchemy today” would be false.

Table	Purpose
`users`	username, email, bcrypt hash
`url_checks`	Every check-url row (anonymous user_id allowed)
`threats`	HIGH/CRITICAL URL rows with heuristic type label
`content_scans`	Multi-channel scans — raw message text is not stored
`extracted_links`	Per-URL children of a content scan
`user_reports` / `domain_reputation`	Community signal, not proof
`scan_feedback`	Helpful yes/no for later review

Planned target

ROADMAP PostgreSQL for concurrent writers, backups, and role separation. `DATABASE_URL` in `config.py` is unused by `connection.py` today. A later phase may add **pgvector** for embedding-similarity of URL/path tokens against curated phishing campaigns — that extension is not in the repository and must not be demoed as live.

- Lock down CORS to the dashboard origin.
- Rotate JWT secret; shorten token TTL; add refresh tokens.
- Global rate limit (Redis or slowapi) in front of scan endpoints.
- Trained supervised model only after curated labels — swap the `ml` channel without changing the API contract.
- Android SMS companion (`mobile_source.py` is a stub, `implemented = False`).

Security posture already in the tree

- bcrypt, JWT, parameterized SQL, SSRF, 5 MB upload cap, report quotas.
- Images not persisted; export report strips secrets.
- User can delete content-scan history.
- pytest with live feed downloads blocked (`PHISHEYE_SKIP_FEED_WARMUP`).

Appendix — Engine file map



Document generated from the PHISHEYE 2.0 codebase. Classification: internal / academic. Not an NTRO deployment certificate.